TradeMind

Production-Grade Multi-Agent Stock Research Platform

Complete full-stack implementation plan — from architecture to deployment — for a 3–4 person team over 6–8 weeks. Built to ship, built to impress.

Document Type	Full Production Implementation Plan
Stack	Next.js · FastAPI · CrewAI/LangGraph · PostgreSQL · FAISS · Redis
Team Size	3–4 members | Part-time | 6–8 weeks
Deployment	Vercel + Render/Railway + Supabase (mostly free tier)
Evaluation	UE23AM343BA2 — LLMs & Applications | Apr 15–23 2025

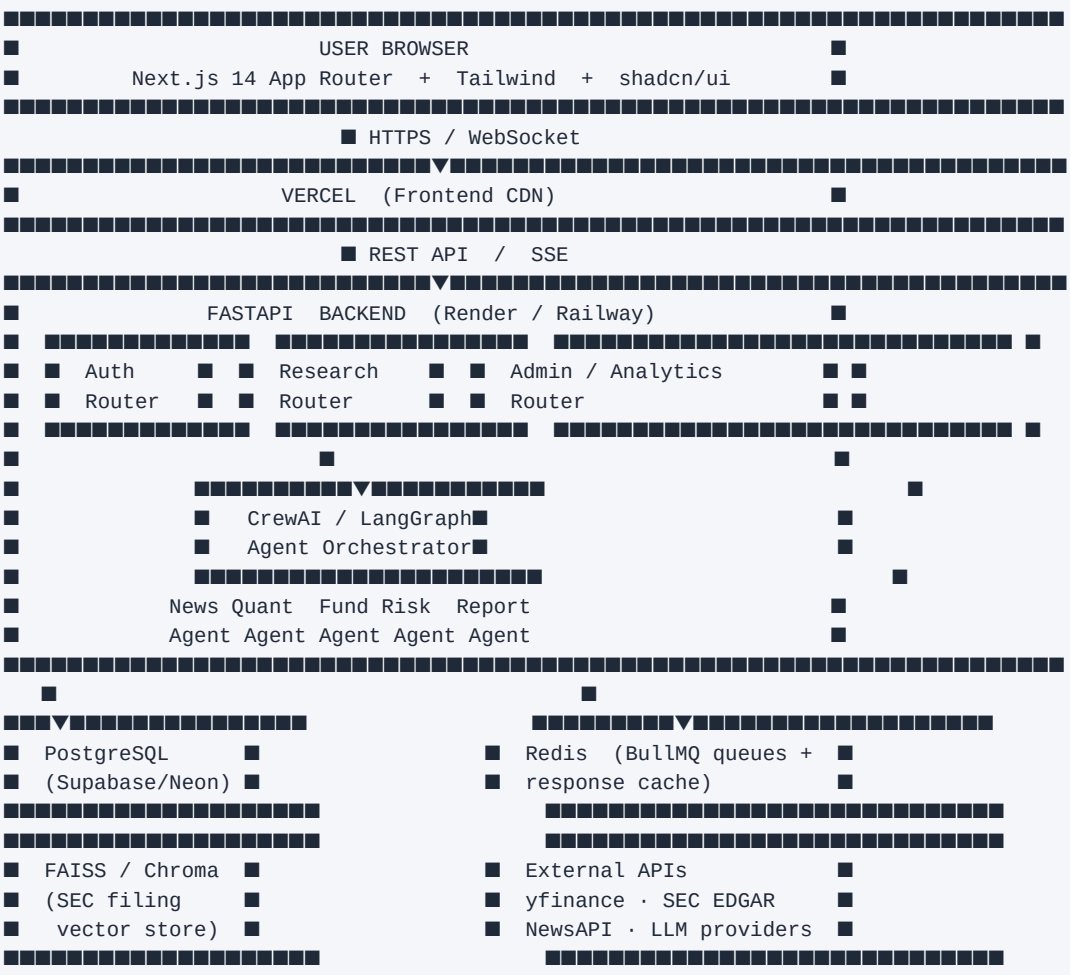
Table of Contents

1. Product Architecture & System Design

1.1 Monolith vs Microservices Recommendation

Recommendation: Modular Monolith for MVP, extract services in Phase 6. Microservices add operational overhead (service discovery, inter-service auth, distributed tracing) that will kill velocity for a small team. Start with a well-structured FastAPI monolith that is internally modular so extraction later is straightforward.

1.2 System Architecture



1.3 Auth Flow

1. User hits /login → Next.js renders login page
2. Choose Email or Google OAuth → NextAuth.js handles OAuth dance
3. On success → JWT (access 15min) + Refresh token (7d) issued
4. Stored: access token in memory, refresh token in httpOnly cookie
5. API calls: Bearer token in Authorization header
6. FastAPI: python-jose verifies JWT, extracts user_id
7. Refresh: silent refresh via /api/auth/refresh on 401

1.4 RAG Pipeline Flow

1. User triggers analysis for ticker e.g. AAPL
2. Fundamental Agent calls SEC EDGAR API → downloads latest 10-K / 10-Q PDF
3. PDFs split into 512-token chunks with 50-token overlap (LangChain TextSplitter)
4. Chunks embedded via sentence-transformers (all-MiniLM-L6-v2, FREE)
5. Embeddings stored in FAISS index (persisted to disk / S3)
6. Agent issues RAG queries: "revenue growth", "debt obligations", "guidance"

```
7. Top-5 similar chunks retrieved per query
8. Chunks injected into LLM prompt as grounding context
9. LLM generates structured analysis citing specific filing sections
10. Result cached in Redis for 24h (same ticker = no re-embed)
```

2. Database Schema

PostgreSQL via Supabase (free tier: 500MB, 2 projects). Use Alembic for migrations. All timestamps in UTC.

```
-- USERS
CREATE TABLE users (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email       TEXT UNIQUE NOT NULL,
    name        TEXT,
    avatar_url  TEXT,
    provider    TEXT DEFAULT 'email',   -- 'email' | 'google'
    created_at  TIMESTAMPTZ DEFAULT now(),
    is_admin    BOOLEAN DEFAULT false
);

-- USER API KEYS (encrypted at rest)
CREATE TABLE user_api_keys (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
    provider    TEXT NOT NULL,          -- 'openai'|'anthropic'|'gemini'|'groq'
    key_enc     TEXT NOT NULL,          -- AES-256 encrypted
    created_at  TIMESTAMPTZ DEFAULT now(),
    UNIQUE(user_id, provider)
);

-- ANALYSIS REQUESTS
CREATE TABLE analysis_requests (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id),
    ticker      TEXT NOT NULL,
    status      TEXT DEFAULT 'queued',  -- queued|running|done|failed
    llm_used    TEXT,
    started_at  TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- AGENT OUTPUTS (one row per agent per request)
CREATE TABLE agent_outputs (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id  UUID REFERENCES analysis_requests(id) ON DELETE CASCADE,
    agent_name  TEXT NOT NULL,
    output_json JSONB,
    tokens_used INT DEFAULT 0,
    latency_ms  INT,
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- RESEARCH REPORTS (final compiled report)
CREATE TABLE research_reports (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id  UUID REFERENCES analysis_requests(id),
    user_id     UUID REFERENCES users(id),
    ticker      TEXT NOT NULL,
    rating      TEXT,                   -- 'BUY'|'HOLD'|'SELL'
    price_target NUMERIC(10,2),
    summary     TEXT,
    full_json   JSONB,
    pdf_url     TEXT,
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- RATE LIMITING
```

```
CREATE TABLE usage_logs (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID REFERENCES users(id),
    action      TEXT,
    tokens_used INT DEFAULT 0,
    created_at  TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_usage_user_day ON usage_logs(user_id, created_at);
```

3. Full Folder Structure

```
trademind/
├── frontend/                        ← Next.js 14 App Router
│   ├── app/
│   │   ├── (auth)/login/page.tsx
│   │   ├── (auth)/register/page.tsx
│   │   ├── dashboard/page.tsx
│   │   ├── research/[ticker]/page.tsx
│   │   ├── profile/page.tsx
│   │   ├── admin/page.tsx
│   │   └── layout.tsx
│   ├── components/
│   │   ├── ui/                      ← shadcn/ui components
│   │   ├── charts/                  ← Recharts wrappers
│   │   ├── report/                  ← Report viewer sections
│   │   └── agents/                  ← Agent status cards
│   ├── lib/
│   │   ├── api.ts                   ← Axios client + interceptors
│   │   ├── auth.ts                  ← NextAuth config
│   │   └── utils.ts
│   ├── hooks/                       ← Custom React hooks
│   ├── store/                       ← Zustand state
│   ├── public/
│   └── next.config.ts
│
├── backend/                         ← FastAPI
│   ├── app/
│   │   ├── main.py                  ← FastAPI app, CORS, middleware
│   │   ├── api/
│   │   │   ├── auth.py
│   │   │   ├── research.py
│   │   │   ├── users.py
│   │   │   └── admin.py
│   │   ├── agents/
│   │   │   ├── crew.py              ← CrewAI orchestration
│   │   │   ├── news_agent.py
│   │   │   ├── quant_agent.py
│   │   │   ├── fundamental_agent.py
│   │   │   ├── risk_agent.py
│   │   │   └── report_agent.py
│   │   ├── tools/
│   │   │   ├── yfinance_tool.py
│   │   │   ├── edgar_tool.py
│   │   │   ├── news_tool.py
│   │   │   └── ta_tool.py
│   │   ├── rag/
│   │   │   ├── embedder.py
│   │   │   ├── retriever.py
│   │   │   └── indexer.py
│   │   ├── report/
│   │   │   ├── generator.py         ← Final report compiler
│   │   │   └── pdf_export.py        ← ReportLab PDF builder
│   │   ├── db/
│   │   │   ├── models.py            ← SQLAlchemy models
│   │   │   ├── session.py
│   │   │   └── migrations/
│   │   ├── core/
│   │   │   ├── config.py            ← Pydantic settings
│   │   │   ├── security.py          ← JWT + key encryption
│   │   │   └── rate_limit.py
│   │   └── tasks/
```

```
■   ■       ■■■ worker.py           ← Celery / ARQ worker
■   ■■■ tests/
■   ■■■ requirements.txt
■   ■■■ Dockerfile
```

4. Development Roadmap — 7 Phases

Total estimated duration: 6–8 weeks part-time (3–4 teammates). Phases 0–3 are critical path. Phases 4–6 can be parallelised.

Phase 0 — Setup & Planning (Days 1–3) — Est. 2–3 days

Tasks	Deliverables	Risks
• Create GitHub monorepo with /frontend and /backend folders • Set up Supabase project → grab DATABASE_URL, run initial migrations • Set up Redis instance on Railway free tier • Configure environment variable templates (.env.example for both) • Bootstrap Next.js 14: npx create-next-app@latest --typescript --tailwind --app • Install shadcn/ui: npx shadcn-ui@latest init • Bootstrap FastAPI: uvicorn + SQLAlchemy + Alembic + pydantic-settings • Set up pre-commit hooks: ruff (Python), eslint + prettier (TS) • Create GitHub Actions CI: lint + type-check on every push • Assign team roles: Backend Lead, Data/RAG, ML/Agents, Frontend/Infra	✓ Repo live ✓ Both apps run locally ✓ DB connected ✓ CI green	■ *None*

Phase 1 — Core Backend (Days 4–10) — Est. 5–7 days

Tasks	Deliverables	Risks
• FastAPI skeleton: CORS, exception handlers, health check endpoint • SQLAlchemy models matching DB schema + Alembic migration v001 • Auth endpoints: POST /auth/register, /auth/login, /auth/refresh, /auth/google • JWT creation (python-jose), bcrypt password hashing • User API key CRUD: POST/GET/DELETE /users/me/api-keys • AES-256 encryption for stored API keys (cryptography library) • Rate limiting middleware: slowapi + Redis backend • Research request endpoint: POST /research/{ticker} → enqueue job • SSE endpoint: GET /research/{request_id}/stream for live progress • ARQ (async task queue) worker setup for background agent jobs • Admin endpoints: GET /admin/logs, GET /admin/usage	✓ Auth working end-to-end ✓ API key storage encrypted ✓ Queue worker running	■ *JWT secret rotation plan* ■ *Redis connection drops on free tier sleeps*

Phase 2 — AI Agents (Days 8–18) — Est. 8–10 days ← CRITICAL PATH

Tasks	Deliverables	Risks

• Install CrewAI, LangChain, sentence-transformers, faiss-cpu, yfinance, pandas-ta • Build yfinance_tool: OHLCV + financials + info dict • Build ta_tool: compute RSI(14), MACD(12,26,9), Bollinger(20,2), 50/200 SMA • Build news_tool: NewsAPI + Yahoo Finance RSS scraper • Build edgar_tool: download 10-K/10-Q via SEC EDGAR XBRL API • RAG pipeline: embedder.py (sentence-transformers) + FAISS indexer • Implement all 5 CrewAI agents with system prompts (see Section 5) • Wire agents into sequential crew in crew.py • Stream agent progress events via SSE back to frontend • Test each agent independently with mock data • Add Redis caching: same ticker within 12h → skip re-embed • Implement LLM provider selector: use user key if present, else Gemini free	✓ All 5 agents return structured JSON ✓ RAG retrieves relevant chunks ✓ Full pipeline < 90s	■ *SEC EDGAR rate limits* ■ *sentence-transformers cold start ~8s on free tier* ■ *Token limits on Gemini free*

Phase 3 — Frontend (Days 12–22) — Est. 8–10 days (parallelise with Phase 2)

Tasks	Deliverables	Risks
• Landing page: hero, features, demo screenshot, CTA • Auth pages: login (email + Google button), register, forgot password • Dashboard: recent reports table, search bar, quick stats • Ticker search: debounced input, autocomplete via Yahoo Finance API • Research page: live agent progress tracker (SSE-powered step indicators) • Report viewer: tabbed sections (News / Technicals / Fundamentals / Risk / Verdict) • Charts: Recharts price chart with MACD/RSI overlays, Bollinger Bands • Sentiment badge, Rating card (BUY/HOLD/SELL with confidence bar) • PDF download button: triggers backend PDF generation • Profile page: name/avatar, API key management form • Framer Motion: page transitions, card reveal animations, loading states • Dark mode via next-themes + Tailwind dark: classes	✓ All pages render ✓ SSE progress updates live ✓ Charts display correctly ✓ PDF downloads	■ *SSE reconnect on tab background* ■ *PDF generation latency on free tier*

Phase 4 — Auth + User Settings (Days 18–24) — Est. 4–5 days

Tasks	Deliverables	Risks
• NextAuth.js: email/password provider + Google OAuth provider • Protect all dashboard routes with middleware.ts • API key settings form: add/remove keys per provider, masked display • Frontend calls backend to store encrypted keys • Profile: avatar upload to Supabase Storage, name update • Admin dashboard: usage table, per-user request counts, error logs • Email verification flow (optional but recommended)	✓ Login/register works ✓ Google OAuth live ✓ API key save/delete works ✓ Protected routes redirect	■ *Google OAuth needs verified domain for production*

Phase 5 — Deployment (Days 22–28) — Est. 4–5 days

Tasks	Deliverables	Risks

• Frontend: push to Vercel, connect GitHub repo, set env vars • Backend: create Render web service, connect GitHub, set env vars • Database: Supabase (already set up) — run migrations in prod • Redis: Railway Redis instance → add REDIS_URL to Render env • CORS: update FastAPI allowed origins to Vercel domain • Set up custom domain (optional: Namecheap ~$1/yr for .tech domain) • Vercel + Render both auto-deploy on main branch push • Smoke test: full end-to-end run on AAPL ticker in production • Set up Sentry for error tracking (free tier)	✓ App live on public URL ✓ End-to-end analysis works in prod ✓ CI/CD deploys automatically	■ *Render free tier sleeps after 15min → first request is slow (add wake-up ping)*

Phase 6 — Polish & Scaling (Days 28–42) — Est. 1–2 weeks (stretch)

Tasks	Deliverables	Risks
• Add LLM comparison mode: run same ticker on 2 LLMs, show diff • Watchlist feature: save tickers, get daily digest email • Report history: browse all past reports • Improve RAG: add hybrid search (BM25 + vector), re-ranker • Add Chroma as alternative to FAISS (better persistence) • Performance: profile slow endpoints, add DB indexes • Accessibility audit (axe-core), keyboard navigation • Write comprehensive test suite (see Section 9) • Record demo video for portfolio / LinkedIn	✓ LLM comparison working ✓ Watchlist live ✓ Test coverage > 70%	*Low*

5. Agent Design — All 5 Agents

■ News Agent

Role	Scrapes and analyses recent financial news, press releases and social sentiment for the ticker.
Tools	NewsAPI (free 100 req/day) · Yahoo Finance RSS · BeautifulSoup scraper · LangChain LLMChain

System Prompt	`You are a financial news analyst. Given the ticker {ticker}, analyse the last 14 days of news.`
	`Return a JSON with: headlines (list), overall_sentiment (Bullish/Bearish/Neutral, 0–100 score),`
	`key_themes (list of 3–5), notable_events (earnings, lawsuits, partnerships), risk_flags (list).`

Memory	Passes headlines + sentiment score to shared CrewAI context.
Fallback	If NewsAPI quota exceeded → fall back to Yahoo Finance RSS. If both fail → mark news_status: unavailable, continue pipeline.

■ Quant Agent

Role	Computes technical indicators from historical OHLCV data and generates a trend signal.
Tools	yfinance (free) · pandas-ta · numpy · Custom TA wrapper tool

System Prompt	`You are a quantitative analyst. Given the following technical indicator values for {ticker}:`
	`RSI={rsi}, MACD={macd}, Signal={signal}, Price vs BB_upper={bb_upper}, BB_lower={bb_lower},`
	`50SMA={sma50}, 200SMA={sma200}, 30d_volatility={vol}.`
	`Return JSON: trend_signal (Strong Buy/Buy/Hold/Sell/Strong Sell), support_level, resistance_level, technical_summary (2 sentences), confidence (0–100).`

Memory	Passes technical signal + key levels to shared context.
Fallback	If yfinance call fails → retry 3x with exponential backoff. If still fails → mark quant_status: unavailable.

■ Fundamental Agent (RAG)

Role	Retrieves and analyses SEC 10-K/10-Q filings using RAG to extract key financial metrics and guidance.
Tools	SEC EDGAR XBRL API (free) · LangChain RAG chain · FAISS vector store · sentence-transformers (local, free)

System Prompt	`You are a fundamental equity analyst. Using the retrieved SEC filing excerpts below, answer:`
	`1. Revenue growth trend (YoY). 2. Gross/operating margins trend.`
	`3. Debt-to-equity ratio and cash position. 4. Management forward guidance.`
	`5. Key risks disclosed in the filing.`
	`Context: {rag_context}`
	`Return structured JSON with each field. Cite the filing section for each claim.`

Memory	Stores filing chunks in FAISS; passes fundamental_summary to shared context.
Fallback	If EDGAR API fails → try alternative SEC full-text search. If filing unavailable → use yfinance .financials DataFrame as fallback.

■■ Risk Agent

Role	Synthesises all prior agent outputs to produce a holistic risk assessment and flag contradictions.

Tools	LangChain LLMChain · Custom contradiction checker prompt · Volatility calculator

System Prompt	`You are a senior risk analyst at a hedge fund. Review the following research data for {ticker}:`
	`News sentiment: {news_sentiment}. Technical signal: {tech_signal}.`
	`Fundamental summary: {fundamental_summary}.`
	`Tasks: 1. Identify any contradictions between signals.`
	`2. Assign overall risk score: Low/Medium/High/Critical.`
	`3. List top 3 bear-case catalysts. 4. List top 2 bull-case catalysts.`
	`5. Estimate 30-day downside risk percentage.`
	`Return structured JSON.`

Memory	Reads all prior agent outputs from shared context; passes risk_assessment forward.
Fallback	If any prior agent output is missing → proceed with available data, flag missing inputs explicitly in output.

■ Report Agent

Role	Compiles all agent outputs into a professional, structured equity research report with final rating.
Tools	LangChain LLMChain · Jinja2 template engine · ReportLab PDF builder

System Prompt	`You are a senior equity research analyst writing a client-facing report for {ticker}.`
	`Using all research data below, produce a professional report with:`
	`1. Executive Summary (3–4 sentences). 2. Investment Rating: BUY / HOLD / SELL + price target.`
	`3. Technical Analysis section. 4. Fundamental Analysis section.`
	`5. Risk Factors section. 6. Investment Thesis (bull/bear case).`
	`Data: {all_research_context}`
	`Tone: professional, precise, evidence-based. No hallucinated figures. Cite sources.`

Memory	Reads full shared context; produces final report JSON + triggers PDF generation.
Fallback	If LLM output is malformed → retry with stricter JSON prompt. If 2nd attempt fails → return raw text with parsing error flag.

6. Free-Tier Cost Optimisation

Service	Free Tier	Strategy	Cost if Exceeded
Gemini Flash	15 RPM, 1M TPD	Default LLM. Cache all responses in Redis 24h	~$0.075/1M tokens
Groq (Llama 3)	30 RPM, 14,400 RPD	Fallback if Gemini quota hit. Fastest inference	Free currently
sentence-transformers	Fully local, free	Run on backend. Cache FAISS index per ticker	$0
Supabase DB	500MB, 2 projects	Prune old usage_logs weekly. Archive old reports	$25/mo Pro
Render Backend	750hr/mo free	Single instance. Add UptimeRobot ping to prevent sleep	$7/mo Starter
Redis (Railway)	$5 credit/mo	TTL all keys. Don't store large blobs. Use for queue + cache only	$5–10/mo
Vercel Frontend	Unlimited deploys	Next.js SSG where possible. ISR for dashboard	$20/mo Pro
NewsAPI	100 req/day free	Cache news for 4h. Supplement with RSS scrapers	$449/mo — avoid!
SEC EDGAR API	10 req/sec, free	Respect rate limits. Cache filings to disk/S3	Free (public API)

Rate Limiting Strategy

- **Global limit:** 5 analysis requests per user per day (free plan), 20 for premium.
- **Redis counter:** INCR user:{id}:daily_count, EXPIREAT midnight. Reject with 429 if exceeded.
- **Token budget:** Track tokens used per request in agent_outputs table. Alert if > 50k tokens/request.
- **LLM fallback chain:** Try user key → Gemini free → Groq free → return cached result if all fail.
- **NewsAPI:** Batch all news calls for a ticker into 1 request. Cache result 4h in Redis.
- **FAISS index:** Persist to disk after first build. Check mtime — only re-embed if filing is > 90 days old.

7. Security & Production Readiness

7.1 API Key Encryption

```
from cryptography.fernet import Fernet

# In config.py — load from environment variable, never hardcode
FERNET_KEY = os.environ['FERNET_SECRET_KEY']  # Generate: Fernet.generate_key()
cipher = Fernet(FERNET_KEY)

def encrypt_key(plaintext: str) -> str:
    return cipher.encrypt(plaintext.encode()).decode()

def decrypt_key(ciphertext: str) -> str:
    return cipher.decrypt(ciphertext.encode()).decode()

# Store encrypted in DB. Decrypt only at request time, never log.
```

7.2 Secrets Management

- **Never commit .env files.** Add to .gitignore. Use .env.example with placeholder values only.
- **Vercel:** All secrets in Project Settings → Environment Variables. Never in next.config.ts.
- **Render:** Secrets in Environment tab. Rotate FERNET_SECRET_KEY and JWT_SECRET quarterly.
- **Database passwords:** Use Supabase connection pooling URL, not direct connection in serverless.
- **Common mistake:** Accidentally logging decrypted API keys in debug mode. Use log sanitiser.

7.3 Auth Best Practices

- Passwords hashed with bcrypt (cost factor 12). Never store plaintext.
- JWT access tokens: 15-minute expiry. Refresh tokens: 7 days, stored in httpOnly Secure cookie.
- CSRF protection: SameSite=Strict on cookies, CSRF token for state-changing requests.
- Rate limit auth endpoints: max 5 failed login attempts per IP per 15 minutes (Redis counter).
- Google OAuth: validate state parameter to prevent CSRF. Use PKCE flow.
- Admin routes: check is_admin flag from DB on every request, not just in JWT.

7.4 Abuse Prevention

- IP-based rate limiting on /research endpoint: max 10 req/hour per IP (unauthenticated).
- Input validation: ticker symbol max 10 chars, alphanumeric only. Reject SQL injection attempts.
- Prompt injection defence: wrap user-provided tickers in explicit delimiters in all LLM prompts.
- Output sanitisation: strip HTML from all LLM outputs before storing or rendering.
- CORS: whitelist only your Vercel domain. Never use wildcard * in production.

8. Deployment Plan

Service	Platform	Plan	Notes
Frontend	Vercel	Free	Auto-deploy from main. Preview deploys on PR branches.
Backend API	Render	Free Web Service	Add UptimeRobot to ping /health every 5min to prevent sleep.
PostgreSQL	Supabase	Free (500MB)	Use connection pooler (port 6543) from serverless contexts.
Redis	Railway	$5 credit/mo	Set maxmemory-policy allkeys-lru. Monitor memory usage.
FAISS Index	Render disk / S3	Free (1GB disk)	Render persistent disk $7/mo or use free Cloudflare R2 (10GB free).
Domain	Namecheap	~$1–12/yr	Get .tech or .dev domain. Configure CNAME to Vercel. SSL auto via Let's Encrypt.
Error Tracking	Sentry	Free (5k events/mo)	Add @sentry/nextjs and sentry-sdk to Python backend.

8.1 Required Environment Variables

```
# Backend (.env)
DATABASE_URL=postgresql://...           # Supabase connection string
REDIS_URL=redis://...                   # Railway Redis
JWT_SECRET_KEY=
JWT_REFRESH_SECRET=
FERNET_SECRET_KEY=
GEMINI_API_KEY=              # Default free LLM
GROQ_API_KEY=                # Fallback free LLM
NEWSAPI_KEY=
EDGAR_USER_AGENT=YourName your@email.com  # Required by SEC

# Frontend (.env.local)
NEXT_PUBLIC_API_URL=https://your-app.onrender.com
NEXTAUTH_URL=https://your-app.vercel.app
NEXTAUTH_SECRET=
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
```

8.2 CI/CD Setup (GitHub Actions)

```
# .github/workflows/deploy.yml
on: push: branches: [main]
jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r backend/requirements.txt
      - run: pytest backend/tests/ -v
  test-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cd frontend && npm ci && npm run type-check && npm run lint
  # Render + Vercel auto-deploy from main on success
```

9. Testing Strategy

9.1 Backend — Unit Tests (pytest)

- **Auth:** test JWT creation/validation, bcrypt hash/verify, refresh token rotation.
- **Encryption:** test API key encrypt/decrypt round-trip, test tampered ciphertext raises error.
- **Rate limiting:** mock Redis, assert 429 on 6th request within window.
- **Agent tools:** mock yfinance/NewsAPI responses, assert correct JSON schema output.
- **RAG:** test embedder produces correct-dimension vectors, test retriever returns top-k results.

9.2 Backend — Integration Tests

- Spin up test PostgreSQL with pytest-asyncio + asyncpg. Run full DB migrations before suite.
- Test full research pipeline with mocked LLM (return fixture JSON, no real API calls).
- Test SSE endpoint: assert progress events arrive in correct order.
- Test admin endpoint returns correct aggregated usage stats.

9.3 Frontend — Component Tests (Vitest + React Testing Library)

- Test TickerSearch component: type input → debounce → API call → render suggestions.
- Test AgentProgressTracker: simulate SSE events → assert steps update correctly.
- Test ReportViewer: pass fixture report JSON → assert all sections render.
- Test API key form: fill form → submit → assert encrypted key saved (mock API).

9.4 End-to-End Tests (Playwright)

- Happy path: register → login → search AAPL → wait for report → download PDF.
- Auth guard: unauthenticated user redirected to /login on dashboard access.
- API key flow: add OpenAI key → run analysis → verify key used (check LLM used field).
- Rate limit: run 6 requests → assert 7th is blocked with friendly error message.

■ Common mistake: writing tests after the fact when the architecture has drifted. Write agent tool tests in Phase 2 as you build. They catch API schema changes early and save hours of debugging.

10. Resume & Demo Impact

10.1 What Makes This Stand Out

Feature	Why Recruiters Care
Multi-agent orchestration	CrewAI/LangGraph with 5 specialised agents — rare in student portfolios
Production RAG pipeline	Full FAISS + sentence-transformers + SEC EDGAR, not a toy chatbot
Live deployed app	Public URL anyone can visit — most projects are just GitHub repos
User API key management	Real product feature showing system design maturity
LLM-agnostic architecture	Works with Gemini, Groq, OpenAI, Anthropic — recruiter wow factor
Encrypted secrets in DB	Shows security awareness beyond hello-world apps
SSE real-time updates	Real-time streaming is a sought-after full-stack skill
Downloadable PDF reports	Tangible, shareable output — show it in a demo without explaining tech

10.2 How to Explain It in Interviews

30-second pitch: "TradeMind is a multi-agent AI platform where five specialised LLM agents simulate a hedge-fund research team. A user types in a stock ticker and within 60 seconds gets a full equity research report including news sentiment, technical analysis, SEC filing insights, risk assessment, and a Buy/Hold/Sell rating — all generated autonomously. It's built on Next.js, FastAPI, CrewAI, RAG with FAISS, and supports multiple LLM backends."

10.3 Demo Preparation Checklist

- Pre-warm the backend: run AAPL analysis 30 minutes before demo so it's cached.
- Have 3 tickers ready: AAPL (stable), TSLA (volatile), a small-cap (shows edge case handling).
- Screenshot the PDF report for slide deck — impressive even when offline.
- Prepare a 2-minute walkthrough: landing → login → search → live progress → report → PDF download.
- Highlight the agent progress tracker screen — most visually impressive part.
- Have the GitHub repo README polished with architecture diagram, tech stack badges, live URL.

11. CLAUDE.md — Project Progress Tracker

Save this as CLAUDE.md in your repo root. Update daily. It's your team's single source of truth.

```
# TradeMind — Project Progress Tracker
> Course: UE23AM343BA2  |  Team: [Your Names]  |  Last Updated: [DATE]

## Project Overview
Multi-agent AI stock research platform. Users enter a ticker, get a
professional equity research report in < 60 seconds.
Live URL: https://trademind.vercel.app  |  API: https://trademind.onrender.com

## Team Roles
| Member | Role           | Primary Responsibility              |
|--------|----------------|-------------------------------------|
| [Name] | Backend Lead   | FastAPI, CrewAI, Auth, DB           |
| [Name] | Data/RAG       | EDGAR pipeline, FAISS, yfinance     |
| [Name] | ML/Agents      | Prompts, LLM eval, benchmarking     |
| [Name] | Frontend/Infra | Next.js UI, Deployment, CI/CD       |

## Milestones
- [ ] Phase 0: Repo + env setup complete
- [ ] Phase 1: Auth + API endpoints working
- [ ] Phase 2: All 5 agents returning structured output
- [ ] Phase 3: Frontend pages built + SSE working
- [ ] Phase 4: Auth + user API key settings live
- [ ] Phase 5: Deployed to Vercel + Render
- [ ] Phase 6: LLM comparison + polish

## Weekly Goals
### Week 1 (Mar 27 – Apr 2)
- [ ] Repo setup, all dependencies installed
- [ ] DB schema migrated on Supabase
- [ ] FastAPI running locally with /health endpoint
- [ ] Next.js running locally with shadcn/ui

### Week 2 (Apr 3 – Apr 9)
- [ ] Auth endpoints complete + tested
- [ ] News Agent returning structured JSON
- [ ] Quant Agent returning technical indicators
- [ ] yfinance tool working reliably

### Week 3 (Apr 10 – Apr 14)  ← CRUNCH WEEK
- [ ] Fundamental Agent + FAISS RAG working
- [ ] Risk Agent + Report Agent complete
- [ ] Full pipeline end-to-end (CLI test)
- [ ] Frontend: dashboard + research page skeleton

### Week 4 (Apr 15 – Apr 23)  ← EVALUATION WEEK
- [ ] App deployed and live
- [ ] Full end-to-end demo recorded
- [ ] LLM comparison results table ready
- [ ] Slide deck done

## Daily Checklist Template
- [ ] Morning standup: what did I do yesterday / what am I doing today / blockers
- [ ] Push code with descriptive commit messages
- [ ] Update this file with progress
- [ ] Test on real ticker before EOD

## Bugs / Issues Tracker
| ID | Description | Status | Owner | Notes |
```

```
|----|-------------|--------|-------|-------|
| B001 | | Open | | |

## Deployment Checklist
- [ ] All env vars set in Vercel + Render
- [ ] DB migrations run in production
- [ ] CORS whitelist updated
- [ ] /health endpoint returns 200
- [ ] Test full pipeline on AAPL in production
- [ ] UptimeRobot ping configured
- [ ] Sentry error tracking active
- [ ] Custom domain configured (optional)

## Stretch Goals
- [ ] LLM comparison mode (GPT-4o vs Claude vs Gemini)
- [ ] Watchlist + email digest
- [ ] Report history browser
- [ ] Hybrid RAG (BM25 + vector)
- [ ] Mobile-responsive UI polish
- [ ] Publish comparison results as research paper
```

Ship it. A deployed, working app beats a perfect unreleased one every time.

TradeMind — Built to impress, designed to ship.